Lecture Computer Vision Chapter 4 – Introduction to Machine Learning

Prof. Dr. Ralph Ewerth
Research Group AI – Multimodal Modelling and Machine Learning
Department of Mathematics and Computer Science
Marburg University & Hessian Center for Artificial Intelligence (hessian.AI)

Marburg University

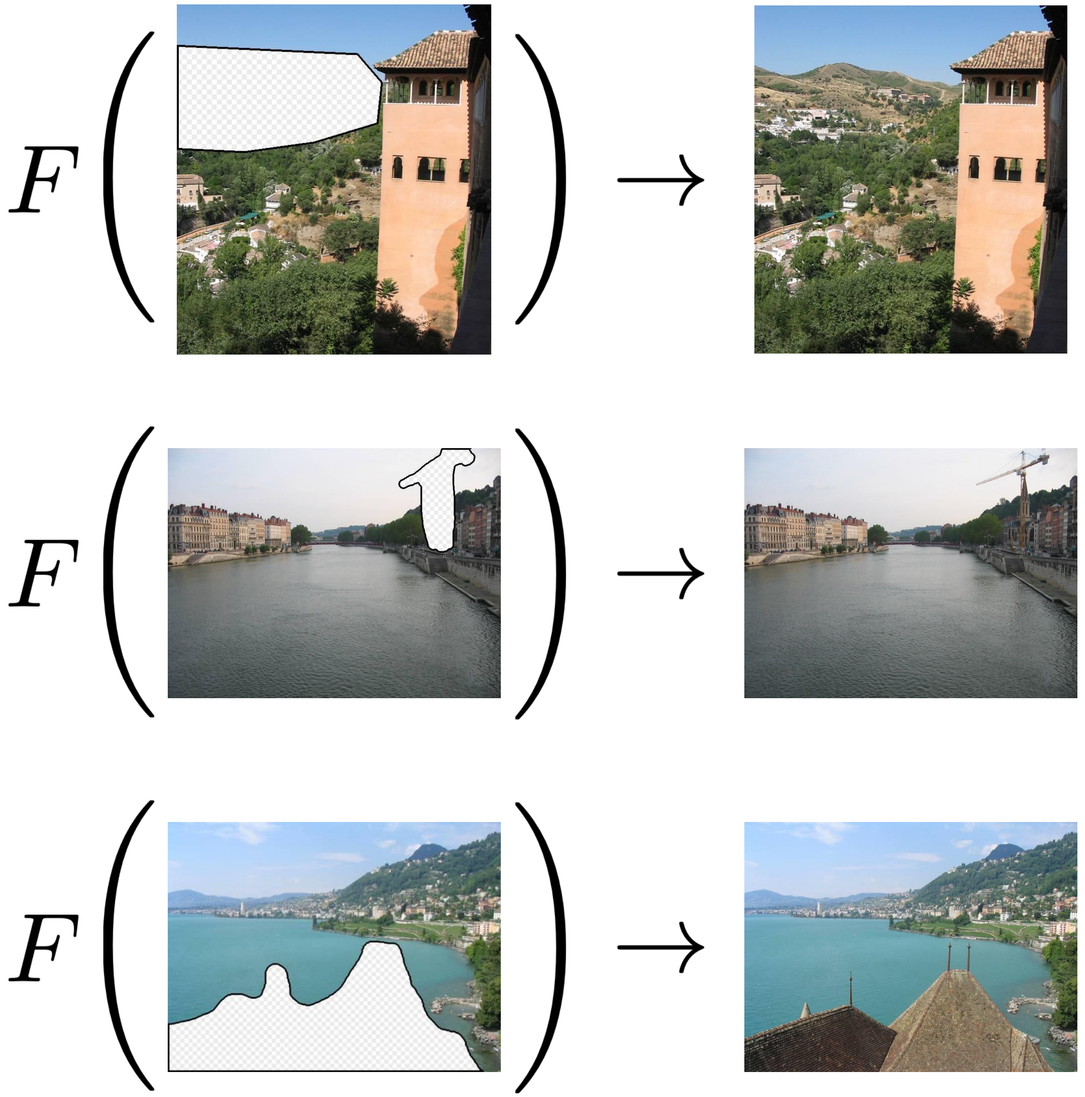
Literature – Text books for this chapter

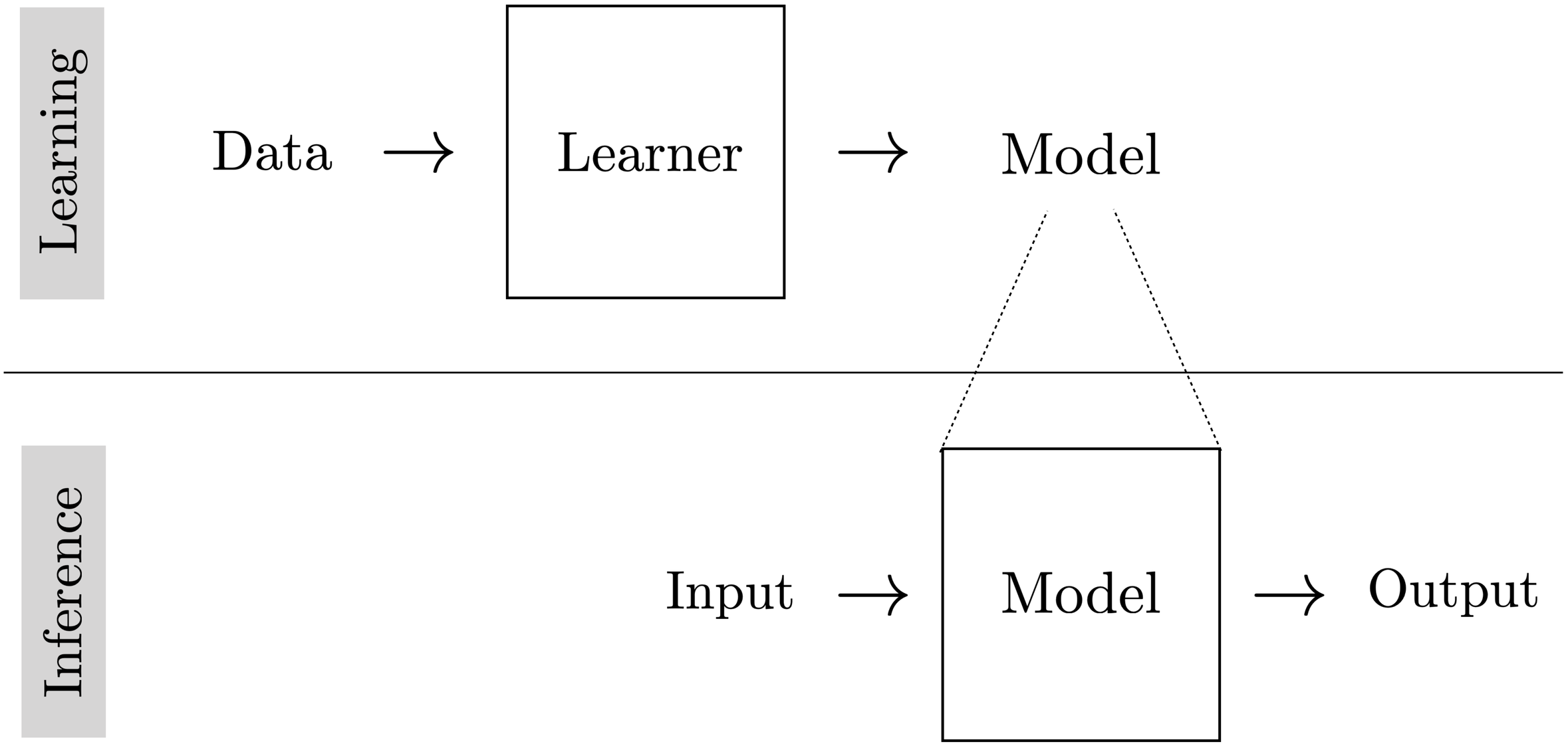
1. Torralba, Antonio, Phillip Isola, & William T. Freeman (2024). *Foundations of Computer Vision*. MIT Press.

2. Szeliski, Richard (2022). *Computer Vision: Algorithms and Applications*. Springer Nature.

Chapter 9

Introduction to Learning




Learning
Data
Learner
Model
Inference
Input
Model
Output

What does ★ do?

$$2 \star 3 = 36$$

$$7 \star 1 = 49$$

$$5 \star 2 = 100$$

$$2 \star 2 = 16$$

Training

$2 \star 3 = 36$

$7 \star 1 = 49$

$5 \star 2 = 100$

$2 \star 2 = 16$

$\rightarrow$ Your brain $\rightarrow$ $x \star y \rightarrow (xy)^2$

Testing

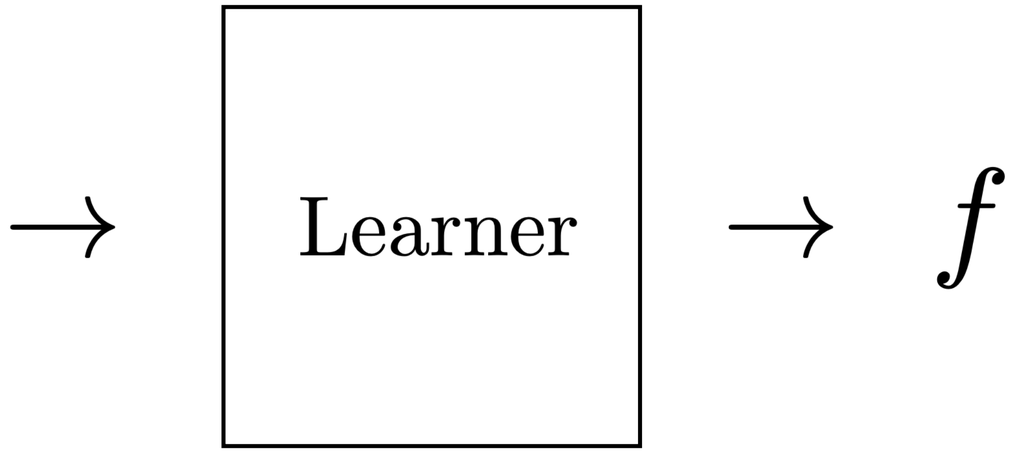
$3 \star 5 \rightarrow$ $x \star y \rightarrow (xy)^2$ $\rightarrow 225$

Learning from examples

(aka supervised learning)

Training data

$\{\texttt{input}: [2, 3], \texttt{output}: 36\}$
$\{\texttt{input}: [7, 1], \texttt{output}: 49\}$
$\{\texttt{input}: [5, 2], \texttt{output}: 100\}$
$\{\texttt{input}: [2, 2], \texttt{output}: 16\}$



Learning from examples

(aka supervised learning)

Training data

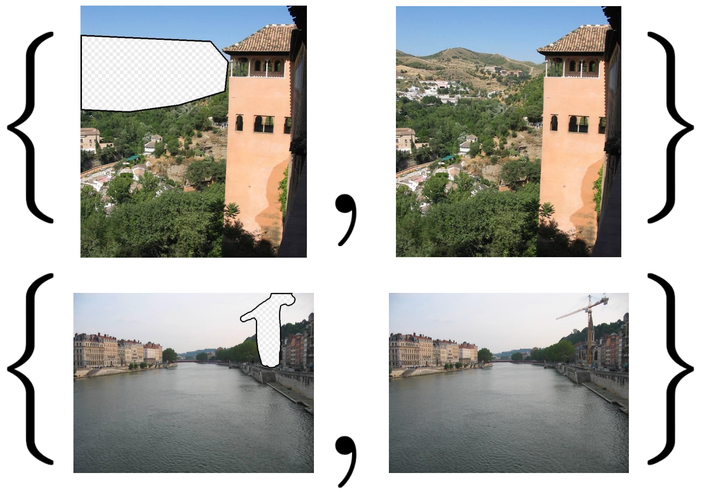
$$\{x^{(1)}, y^{(1)}\}$$
$$\{x^{(2)}, y^{(2)}\} \rightarrow \boxed{\text{Learner}} \rightarrow f : X \rightarrow Y$$
$$\{x^{(3)}, y^{(3)}\}$$
$$\ldots$$

Learning from examples

(aka supervised learning)

Training data

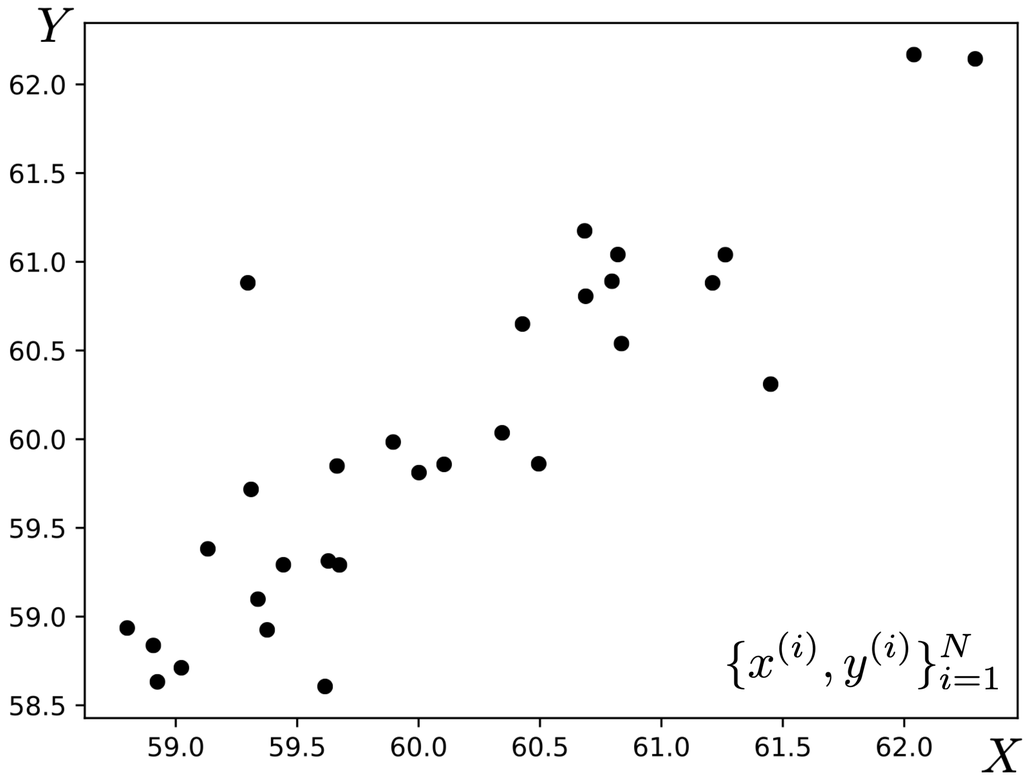
X Y



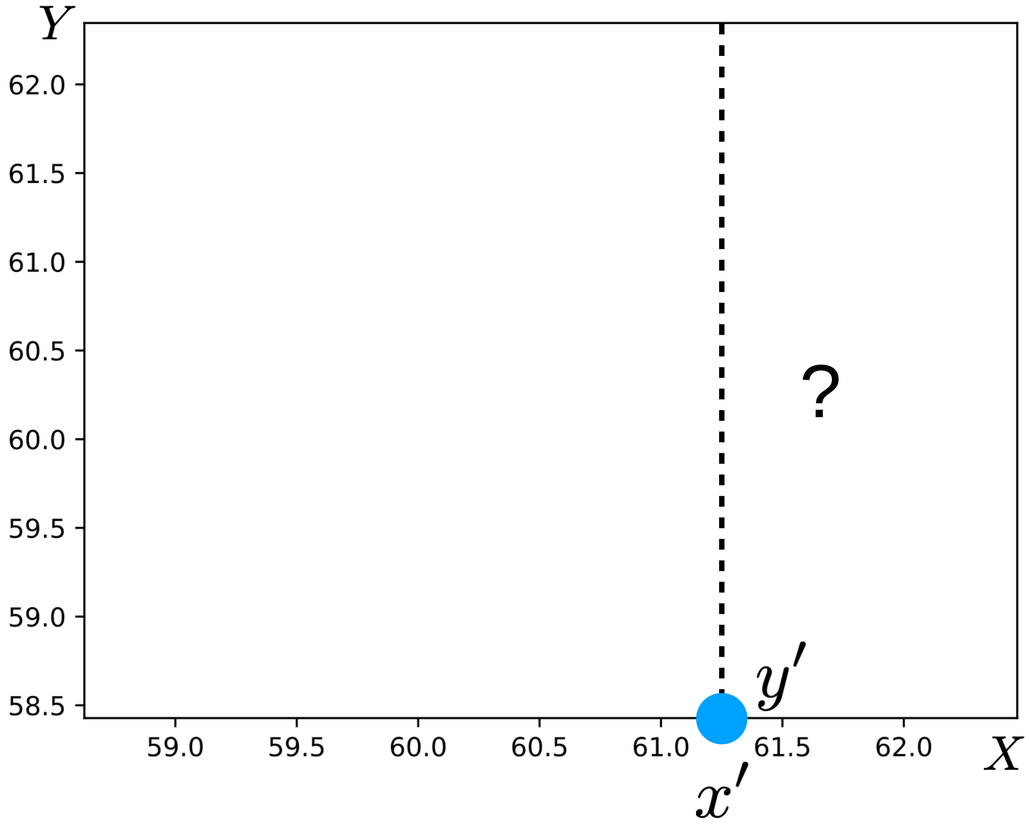
$\rightarrow$ Learner $\rightarrow$ $f : X \rightarrow Y$

Case study #1: Linear least squares

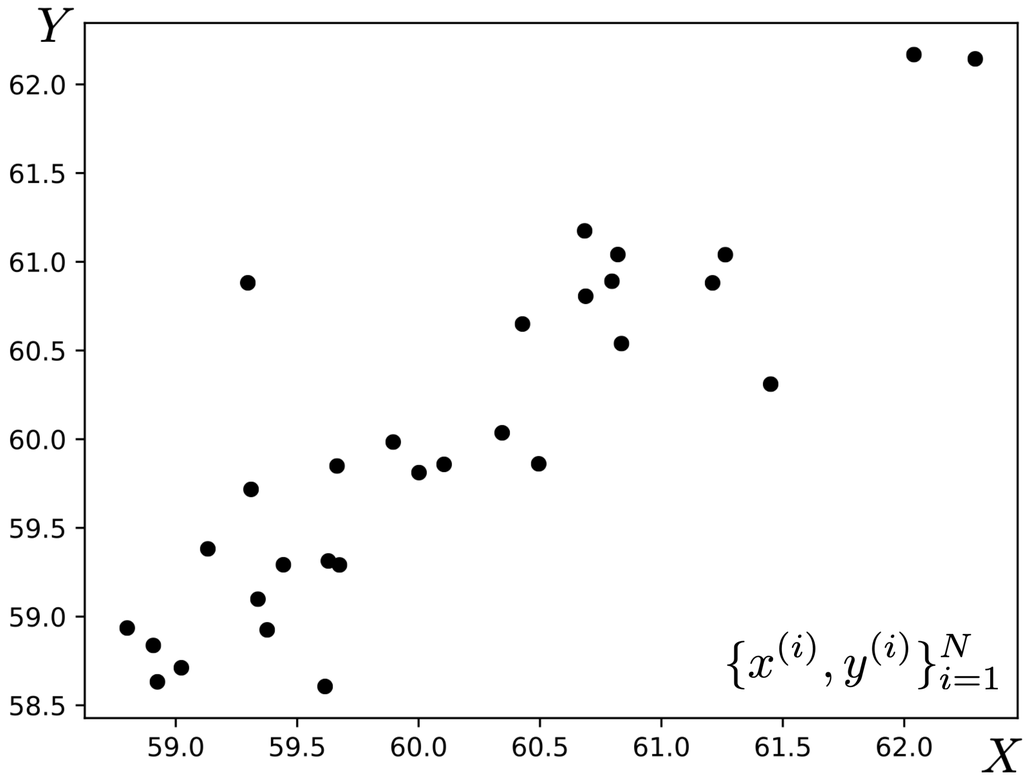
Training data



Test query



Training data

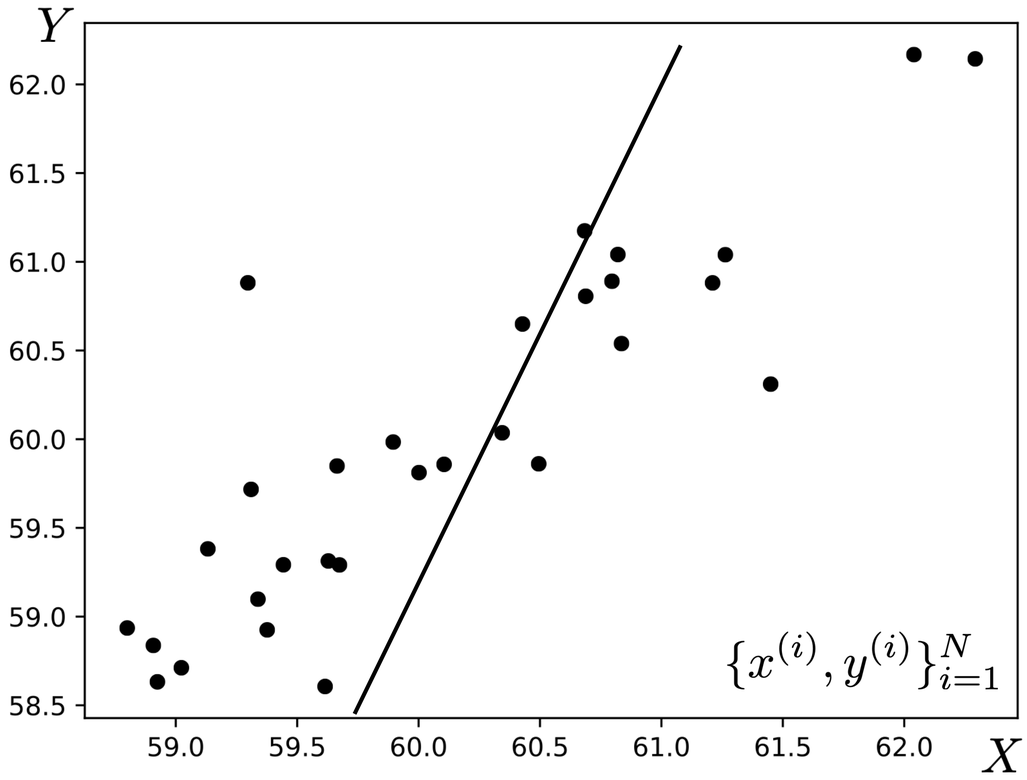


$\longrightarrow$ Learner $\longrightarrow$ $f_\theta(x) = \theta_1 x + \theta_0$

Hypothesis space

The relationship between X and Y is roughly linear: $y \approx \theta_1 x + \theta_0$

Training data

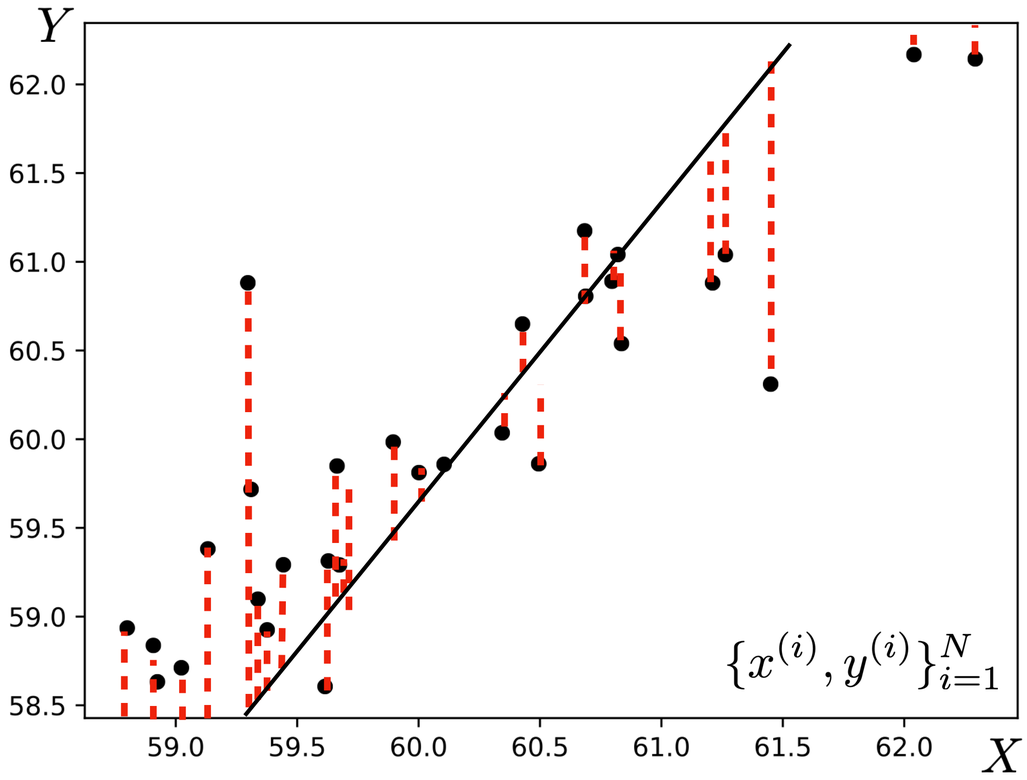


Search for the parameters, $\theta = \{\theta_0, \theta_1\}$ that best fit the data.

$$f_\theta(x) = \theta_1 x + \theta_0$$

Best fit in what sense?

Training data



Search for the parameters, $\theta = \{\theta_0, \theta_1\}$ that best fit the data.

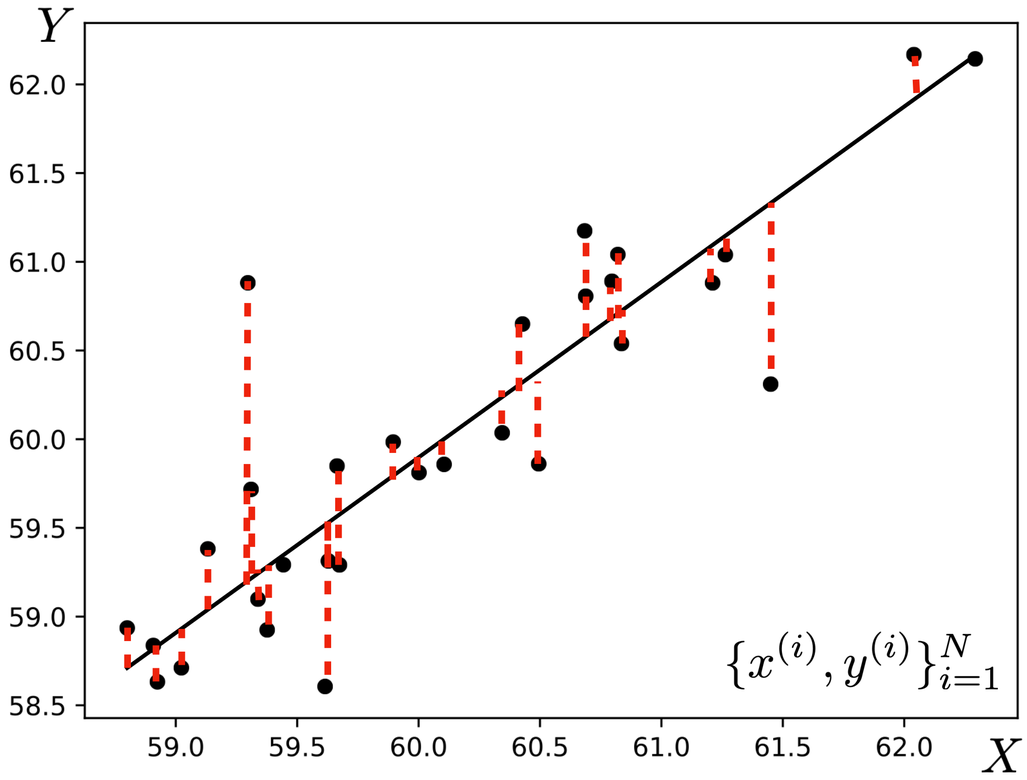
$$f_\theta(x) = \theta_1 x + \theta_0$$

Best fit in what sense?

The least-squares objective (aka loss) says the best fit is the function that minimizes the squared error between predictions and target values:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2 \quad \hat{y} \equiv f_\theta(x)$$

Training data



Search for the parameters, $\theta = \{\theta_0, \theta_1\}$ that best fit the data.

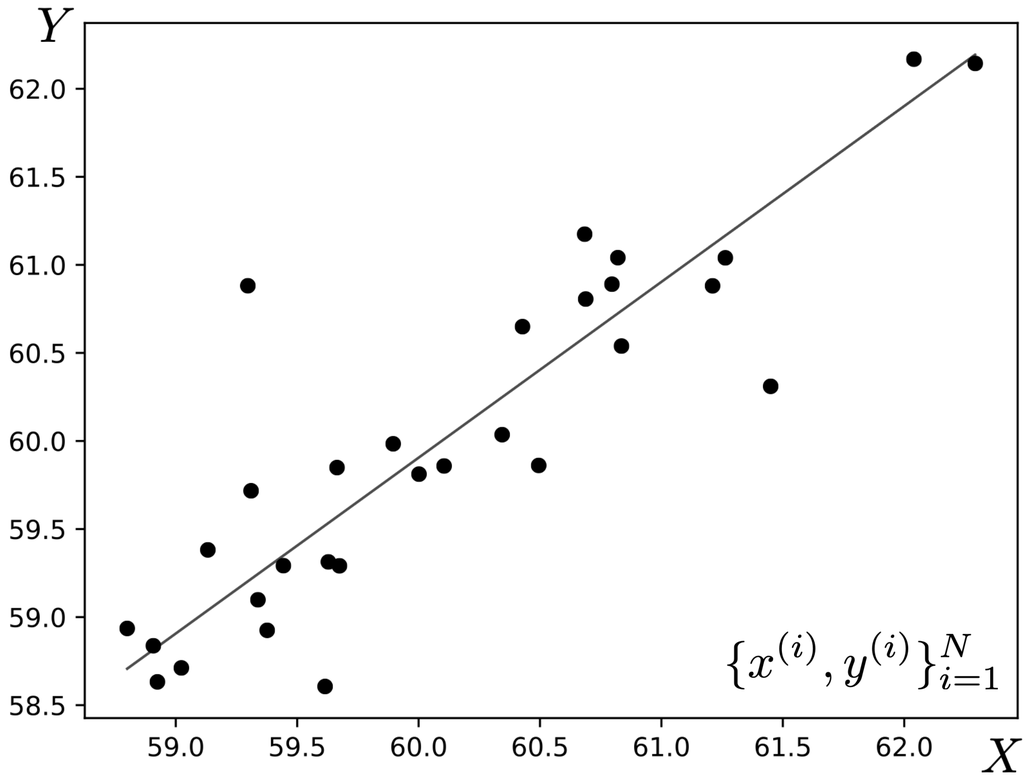
$$f_\theta(x) = \theta_1 x + \theta_0$$

Best fit in what sense?

The least-squares objective (aka loss) says the best fit is the function that minimizes the squared error between predictions and target values:

$$\mathcal{L}(\hat{y}, y) = (\hat{y} - y)^2 \quad \hat{y} \equiv f_\theta(x)$$

Training data

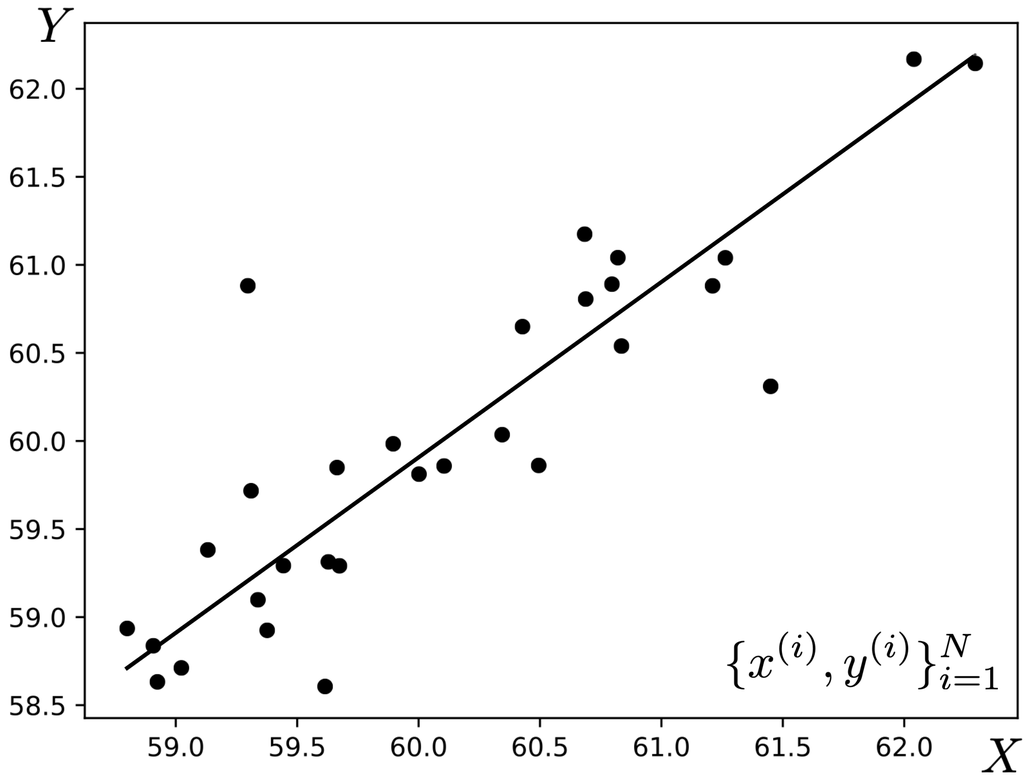


Complete learning problem:

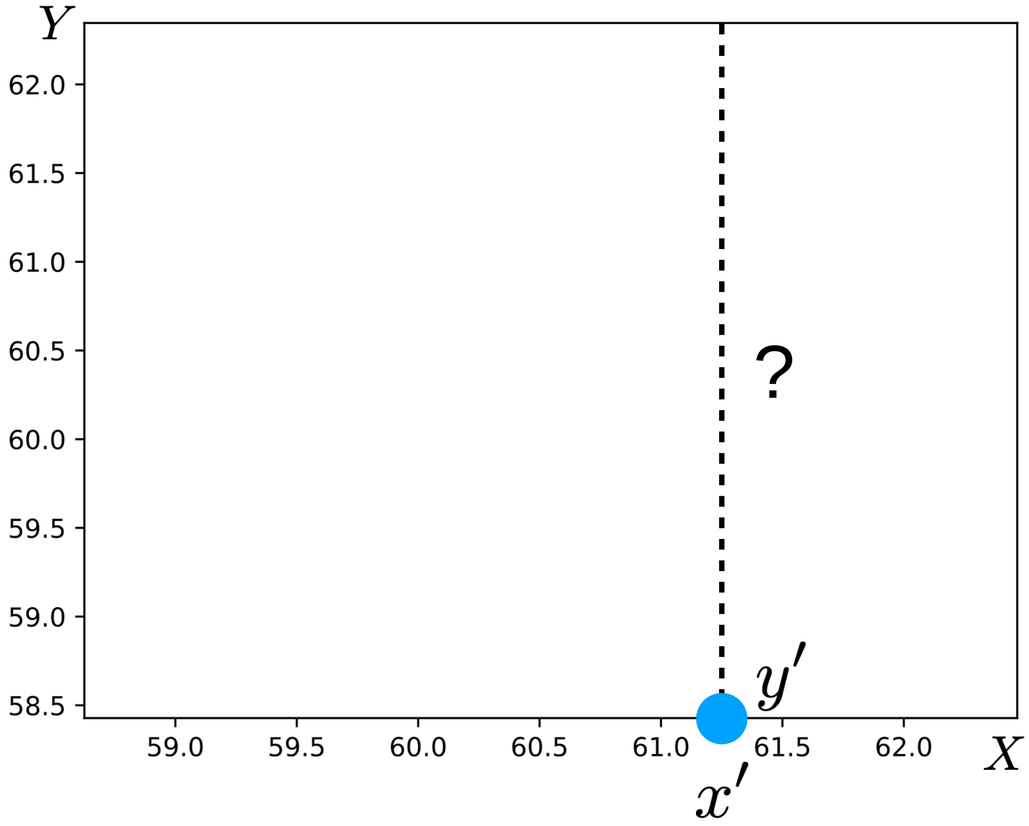
$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{N} (f_\theta(x^{(i)}) - y^{(i)})^2$$

$$= \arg\min_{\theta} \sum_{i=1}^{N} (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

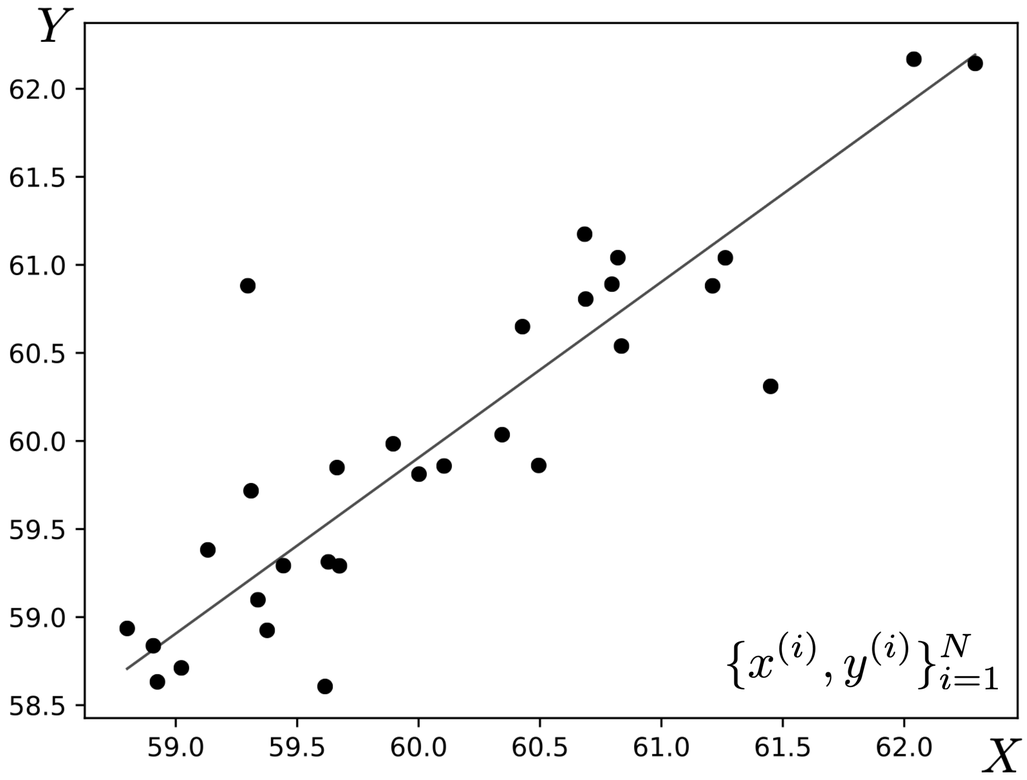
Training data



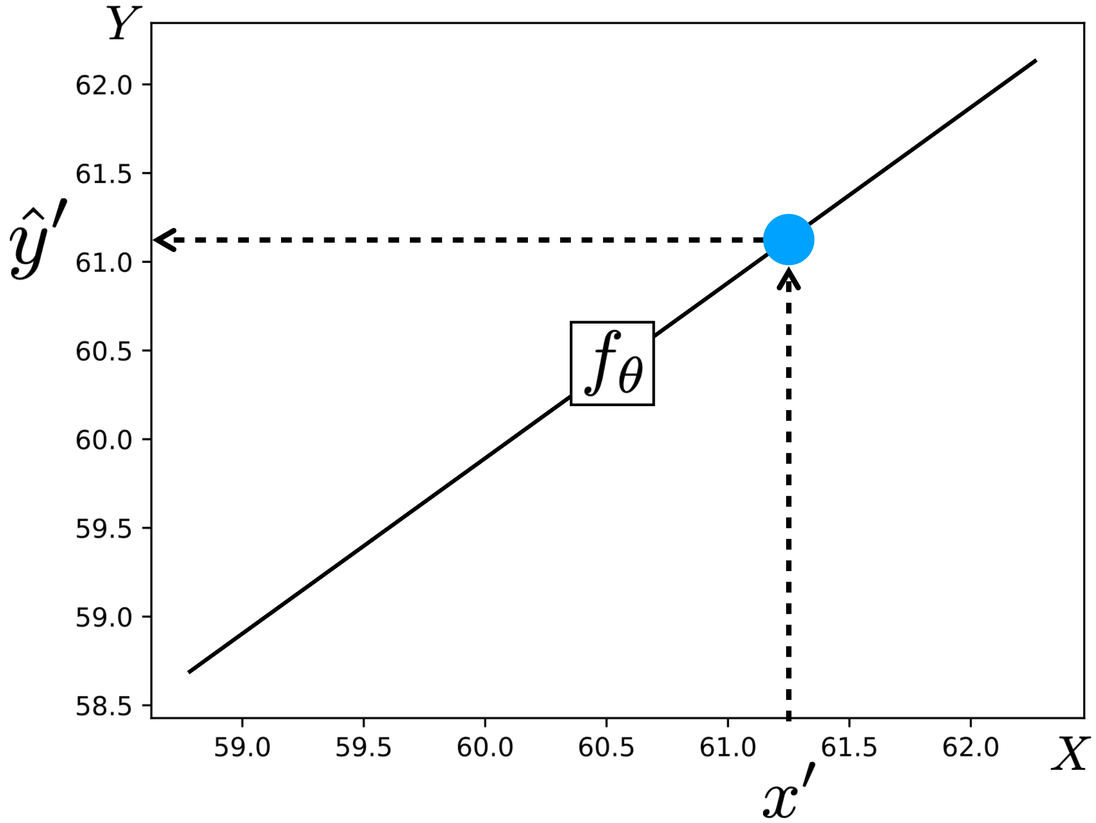
Test query



Training data



Test query

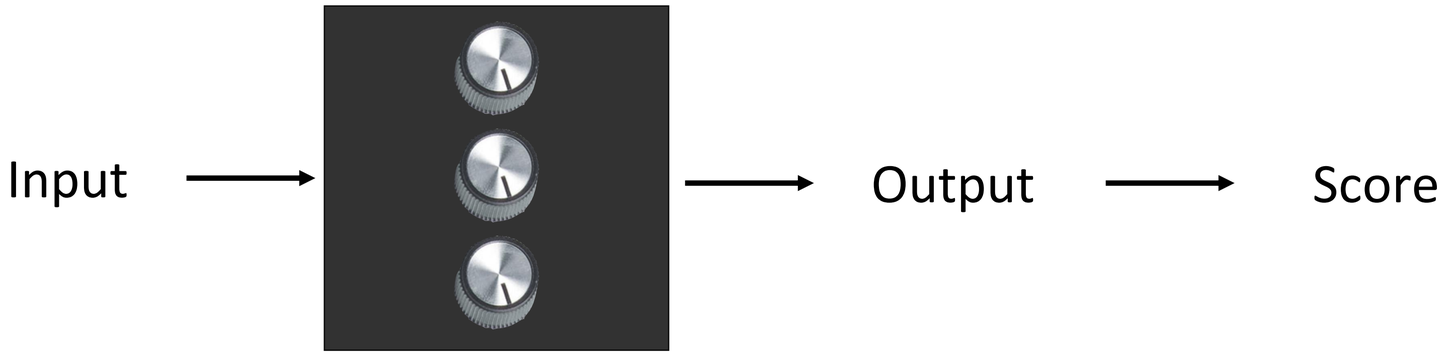


$$x' \dashrightarrow \boxed{f_\theta} \dashrightarrow \hat{y}'$$

How to minimize the objective w.r.t. θ?

$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{N} (f_\theta(x^{(i)}) - y^{(i)})^2$$

Use an optimizer!



Machine with knobs

How to minimize the objective w.r.t. θ?

In the linear case:

Learning problem

$$\theta^* = \arg\min_\theta \sum_{i=1}^{N} (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

$$J(\theta) = \sum_{i=1}^{N} (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

$$= (\mathbf{y} - \mathbf{X}\theta)^T (\mathbf{y} - \mathbf{X}\theta)$$

$$\mathbf{X} = \begin{bmatrix} 1 & x^{(1)} \\ 1 & x^{(2)} \\ \vdots & \vdots \\ 1 & x^{(N)} \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(N)} \end{bmatrix} \quad \theta = \begin{bmatrix} \theta_0 \\ \theta_1 \end{bmatrix}$$

$$\theta^* = \arg\min_\theta J(\theta)$$

$$\frac{\partial J(\theta)}{\partial \theta} = 0$$

$$\frac{\partial J(\theta)}{\partial \theta} = 2(\mathbf{X}^T\mathbf{X}\theta - \mathbf{X}^T\mathbf{y})$$

$$2(\mathbf{X}^T\mathbf{X}\theta^* - \mathbf{X}^T\mathbf{y}) = 0$$

$$\mathbf{X}^T\mathbf{X}\theta^* = \mathbf{X}^T\mathbf{y}$$

Solution

$$\boxed{\theta^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}}$$

Empirical Risk Minimization

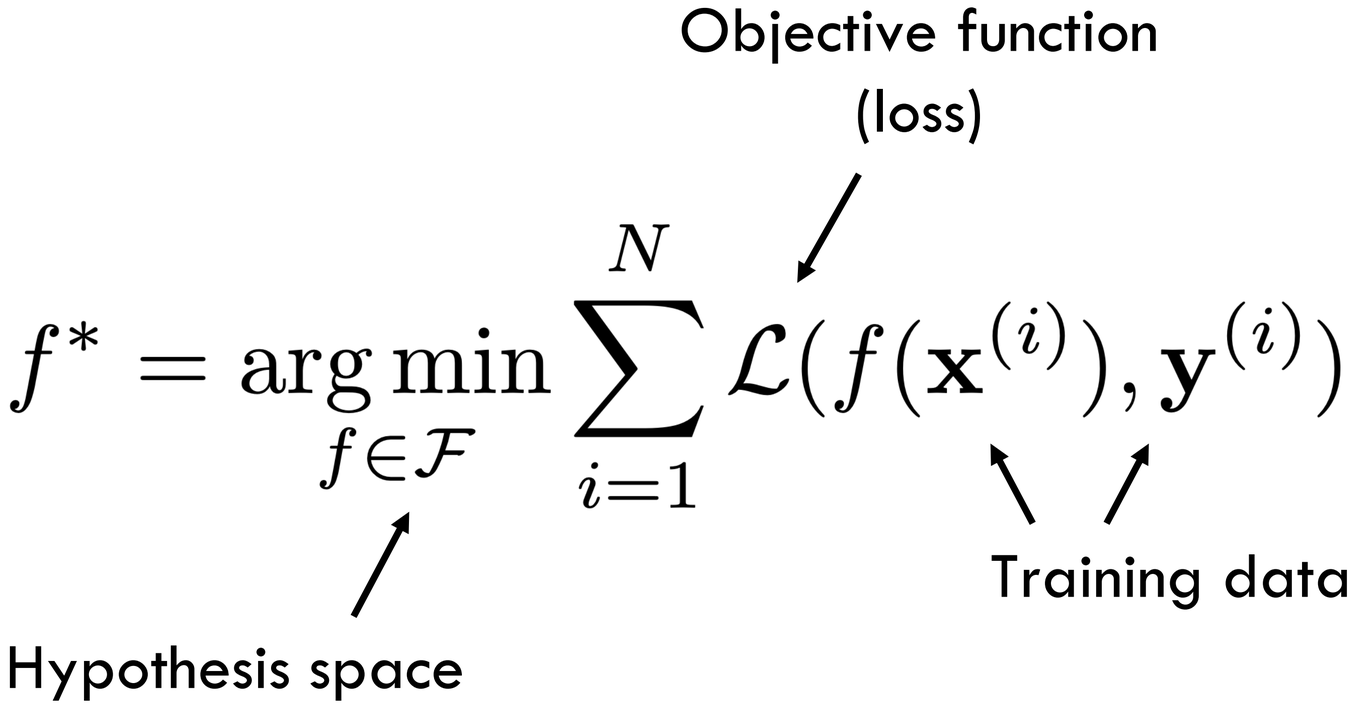
(formalization of supervised learning)

Linear least squares learning problem

$$\theta^* = \arg\min_{\theta} \sum_{i=1}^{N} (\theta_1 x^{(i)} + \theta_0 - y^{(i)})^2$$

Empirical Risk Minimization

(formalization of supervised learning)



Case study #1: Linear least squares

Data

$$\{x^{(i)}, y^{(i)}\}_{i=1}^{N} \rightarrow$$

Learner

Objective

$$\mathcal{L}(f_\theta(x), y) = (f_\theta(x) - y)^2$$

Hypothesis space

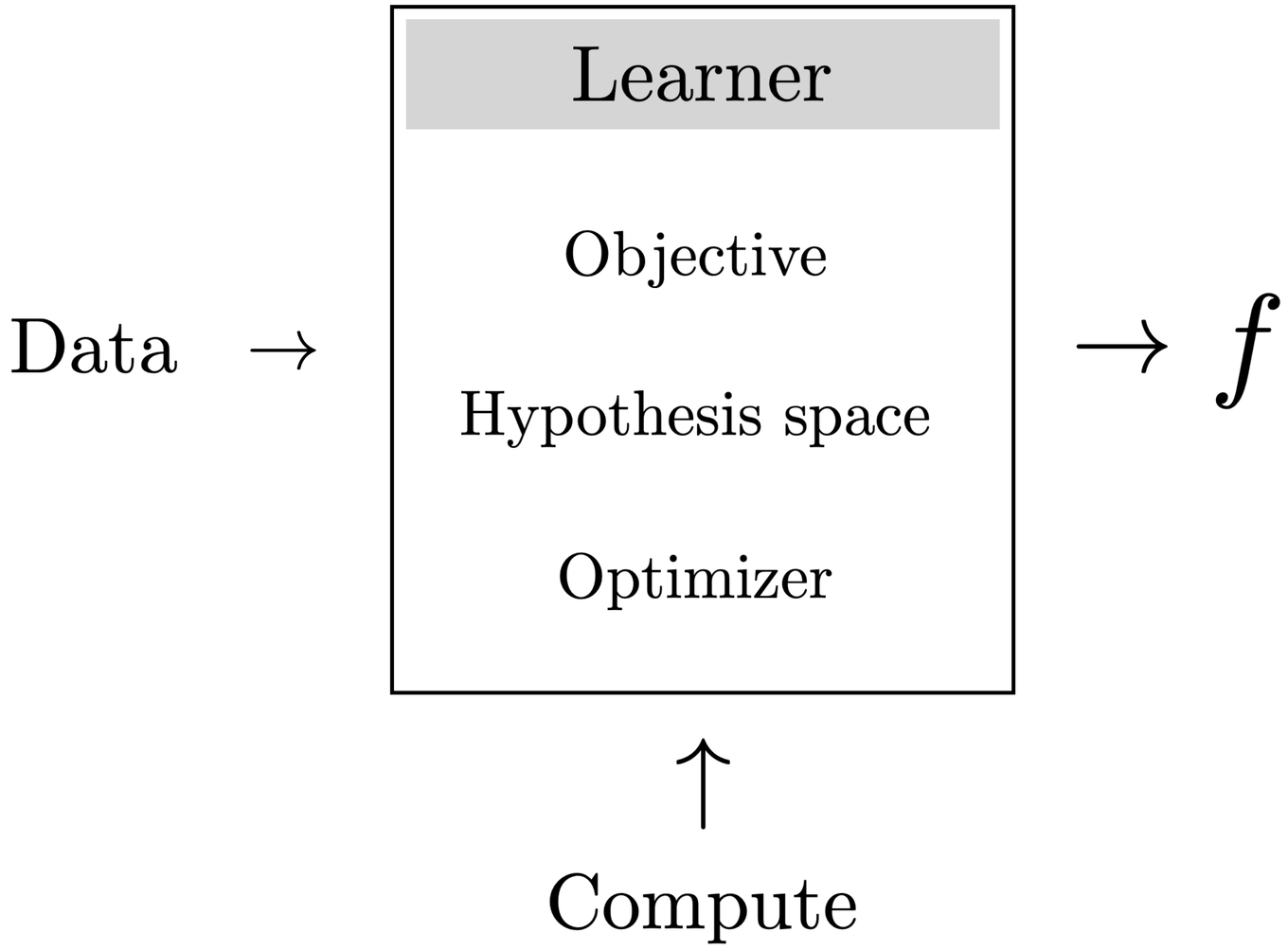
$$f_\theta(x) = \theta_1 x + \theta_0$$

Optimizer

$$\theta^* = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

$$\rightarrow f_\theta$$

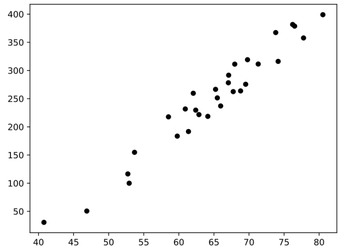
Key Ingredients



Example 1: Linear least squares

Training

Data



Learner

y

f_θ

Testing

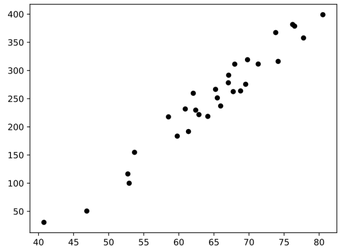
Input

Output

Example 2: Python program induction

Training

Data



→ Learner →

```
def predict(x):
    y = 9.95*x - 395.43
    return y
```

Testing

Input →

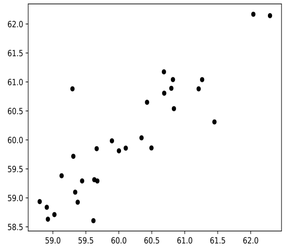
```
def predict(x):
    y = 9.95*x - 395.43
    return y
```

→ Output

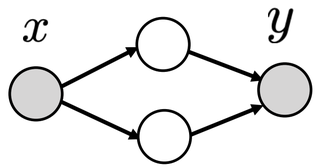
Example 3: Deep learning

Training

Data

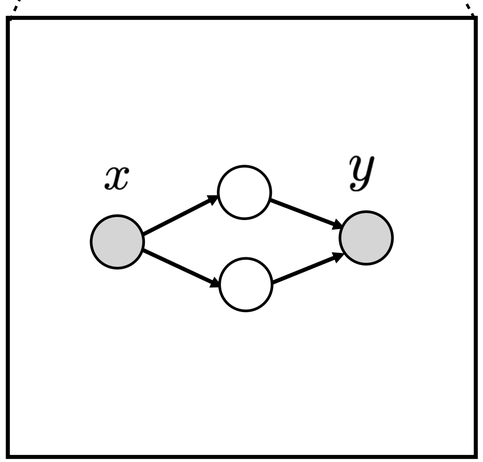


→ Learner →



Testing

Input →



→ Output

Learning for vision

Big questions:

1. How do you represent the input and output?

2. What is the objective?

3. What is the hypothesis space? (e.g., linear, polynomial, neural net?)

4. How do you optimize? (e.g., gradient descent, Newton's method?)

5. What data do you train on?

Case study #2: Image classification

1. How do you represent the input and output?

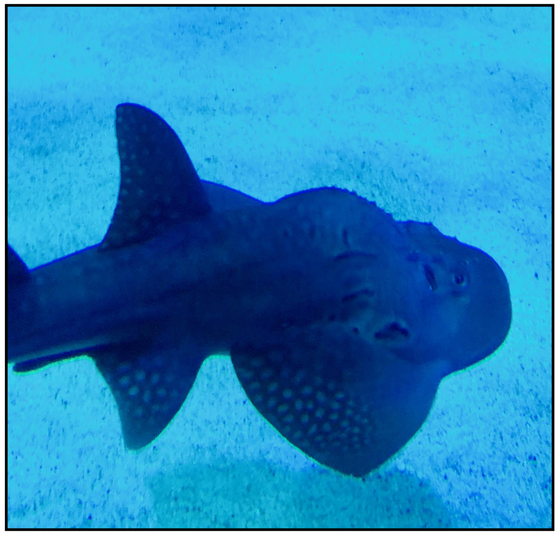
2. What is the objective?

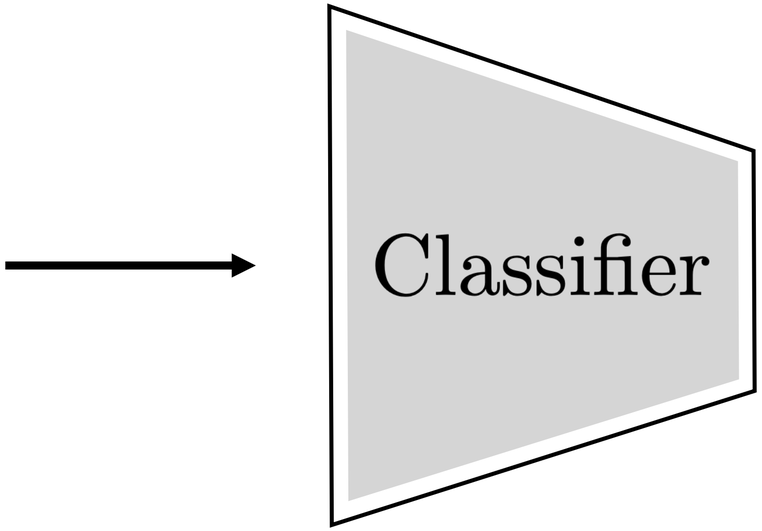
3. Assume hypothesis space is sufficienly expressive

4. Assume we optimize perfectly

5. Assume we train on exactly the data we care about

Image classification





Guitarfish

image x

label y

Image classification

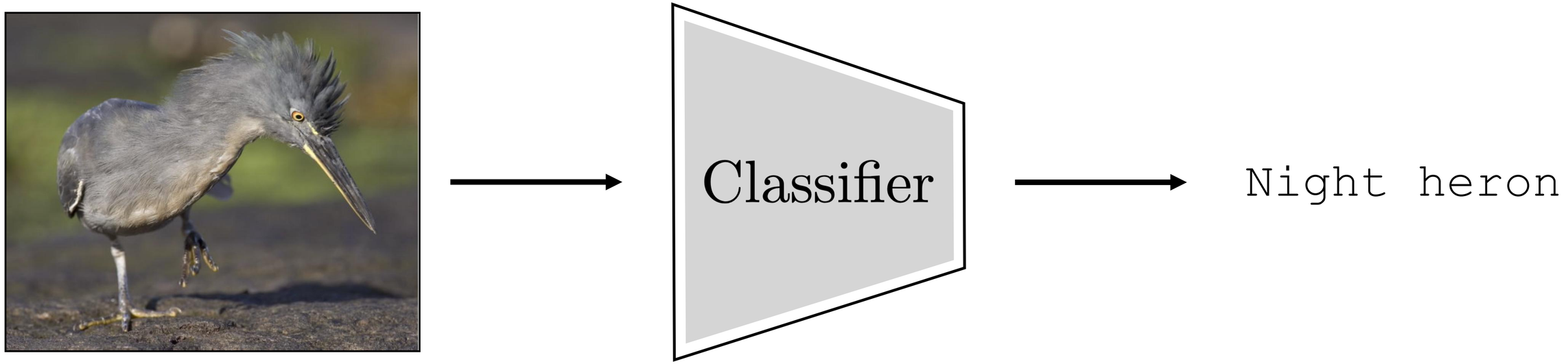
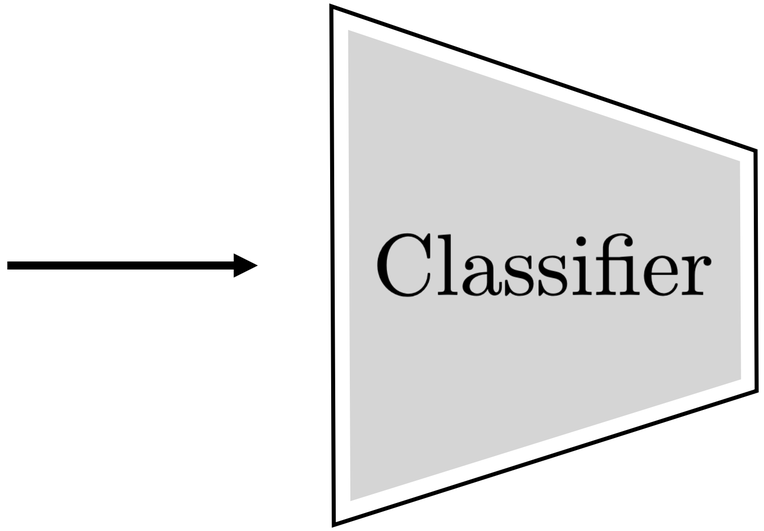


image x

label y

Image classification

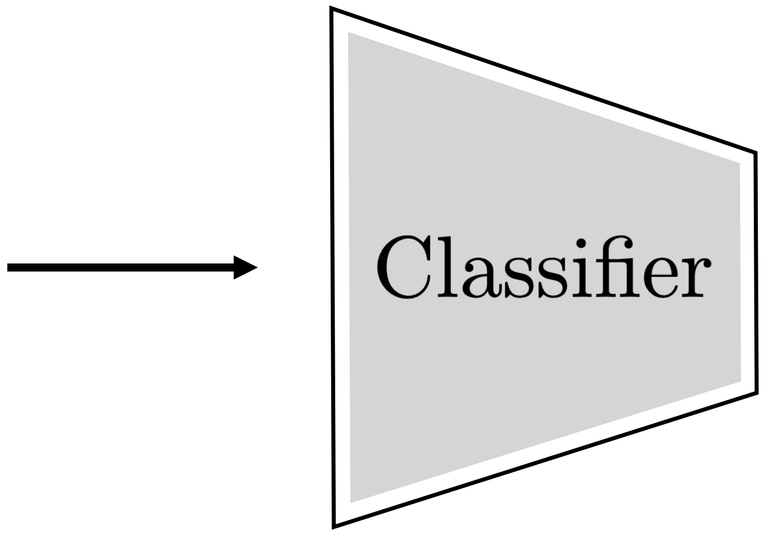


Bumble bee

image x

label y

Image classification

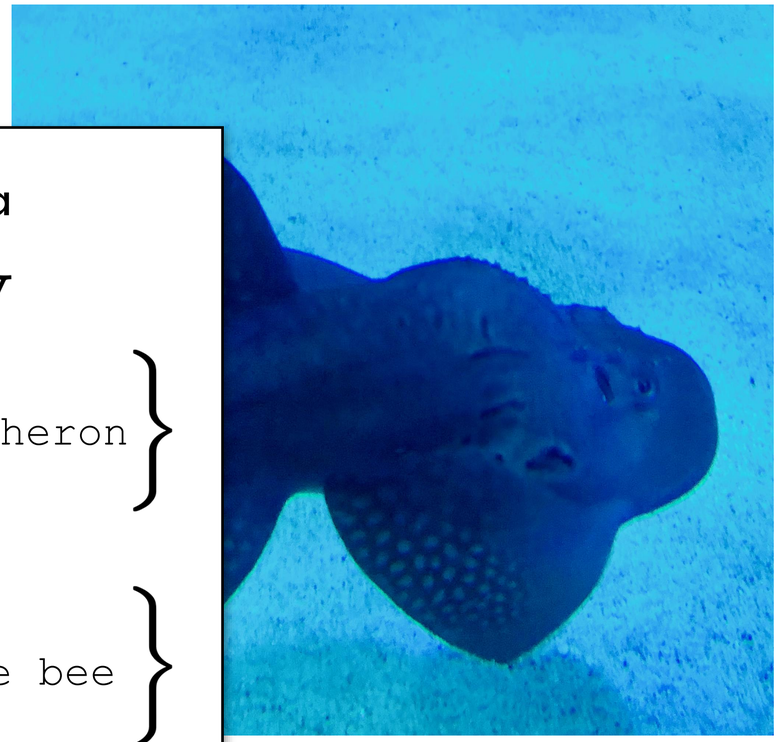


Car

image x

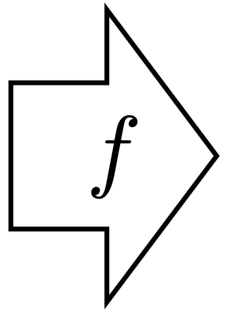
label y

$\mathbf{x}$



Training data

$\mathbf{x}$	$\mathbf{y}$
	Night heron
	Bumble bee
	Car
⋮	



$\mathbf{y}$

Guitarfish

$$\arg\min_{f \in \mathcal{F}} \sum_{i=1}^{N} \mathcal{L}(f(\mathbf{x}^{(i)}), \mathbf{y}^{(i)})$$

How to represent class labels?

One-hot vector

Training data

x	y
	Night heron
	Bumble bee
	Car
⋮	

Training data

x	y
	1
	2
	3
⋮	

Training data

x	y
	[1,0,0]
	[0,1,0]
	[0,0,1]
⋮	

What should the loss be?

0-1 loss (number of misclassifications)

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \mathbb{1}(\hat{\mathbf{y}} = \mathbf{y})$$ ⟵ discrete, NP-hard to optimize!

Cross entropy

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = H(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$ ⟵ continuous, differentiable, convex

Ground truth label

y

x

f

[0,0,0,0,0,1,0,0,…]

x

f

Ground truth label

y

dolphin

cat

night heron

angel fish

bumble bee

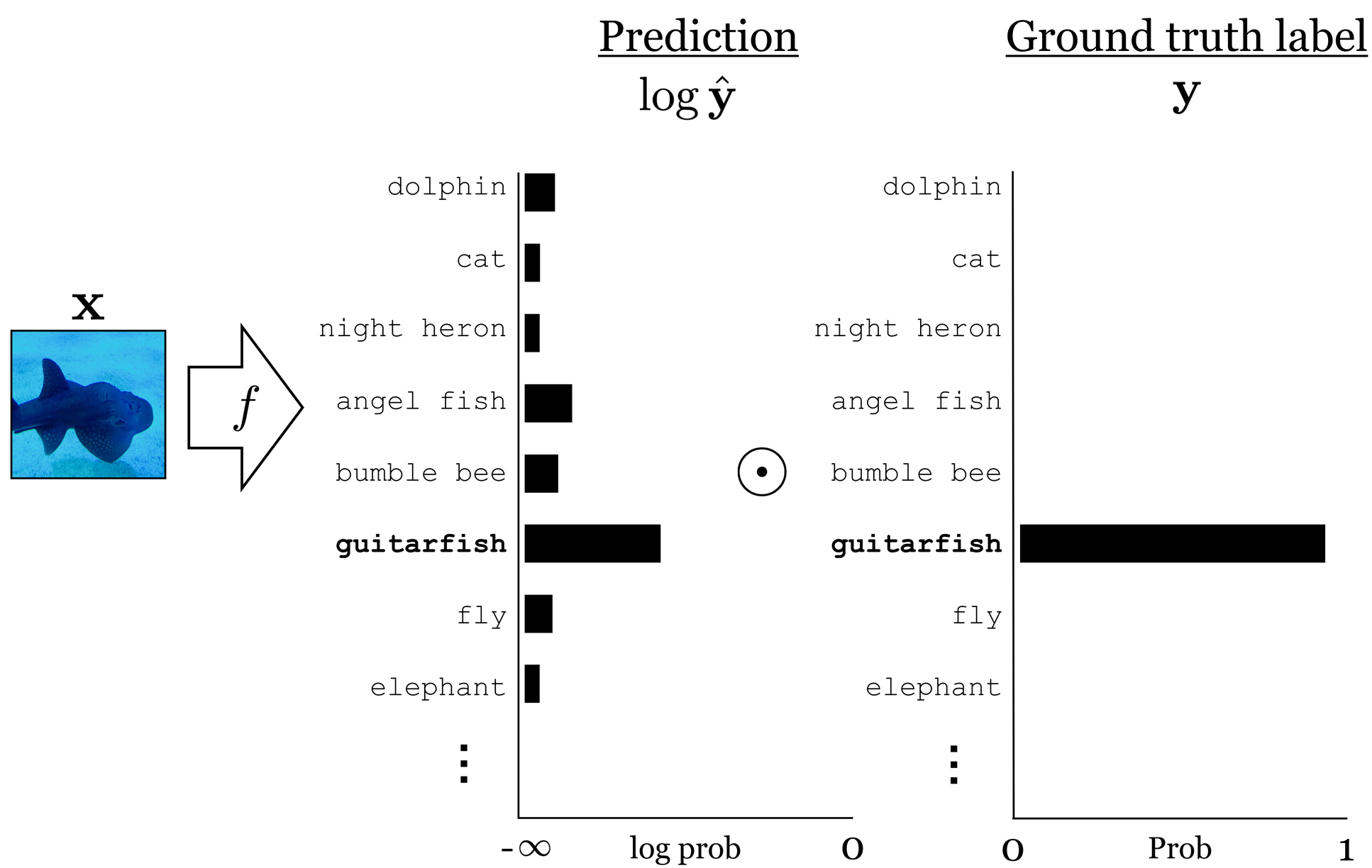
guitarfish

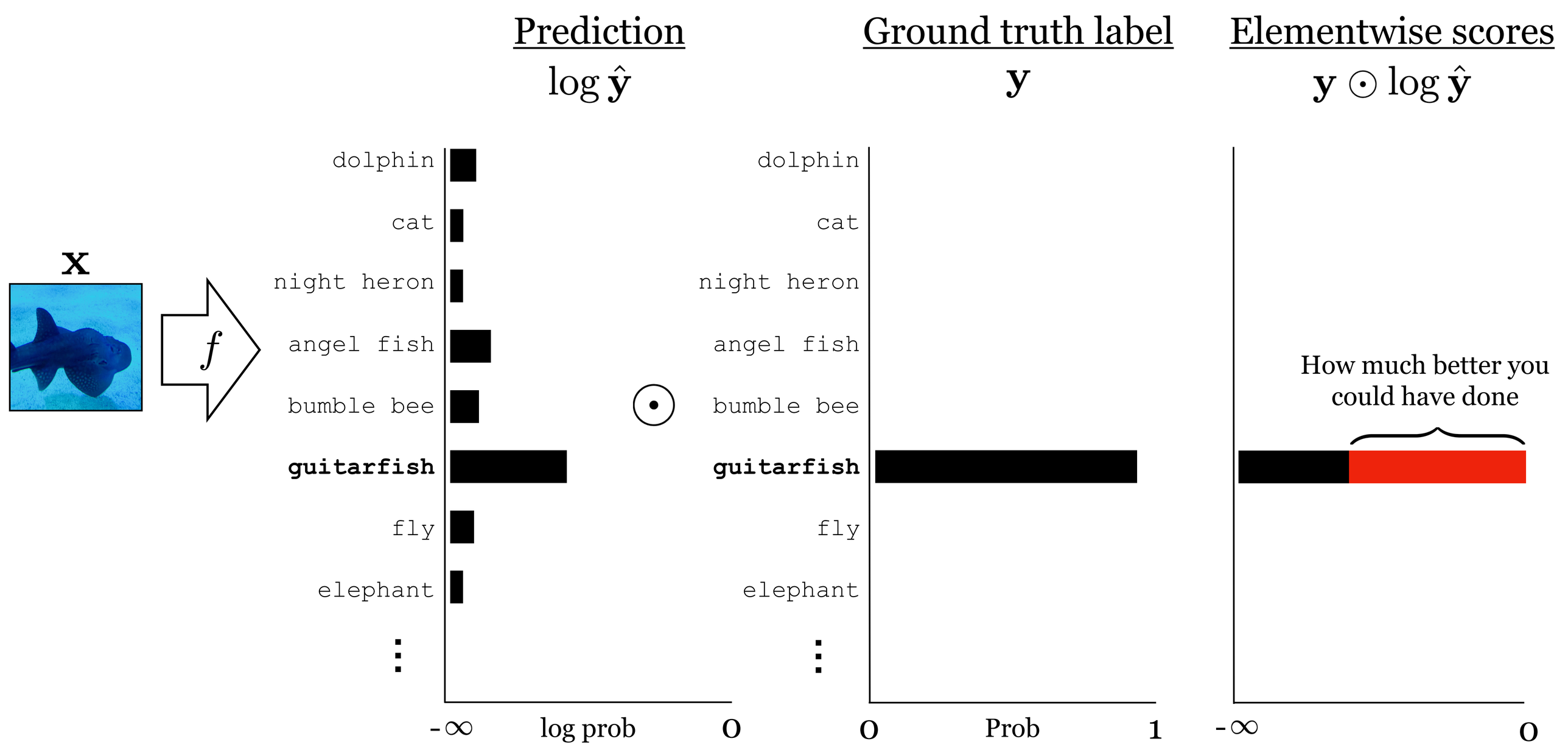
fly

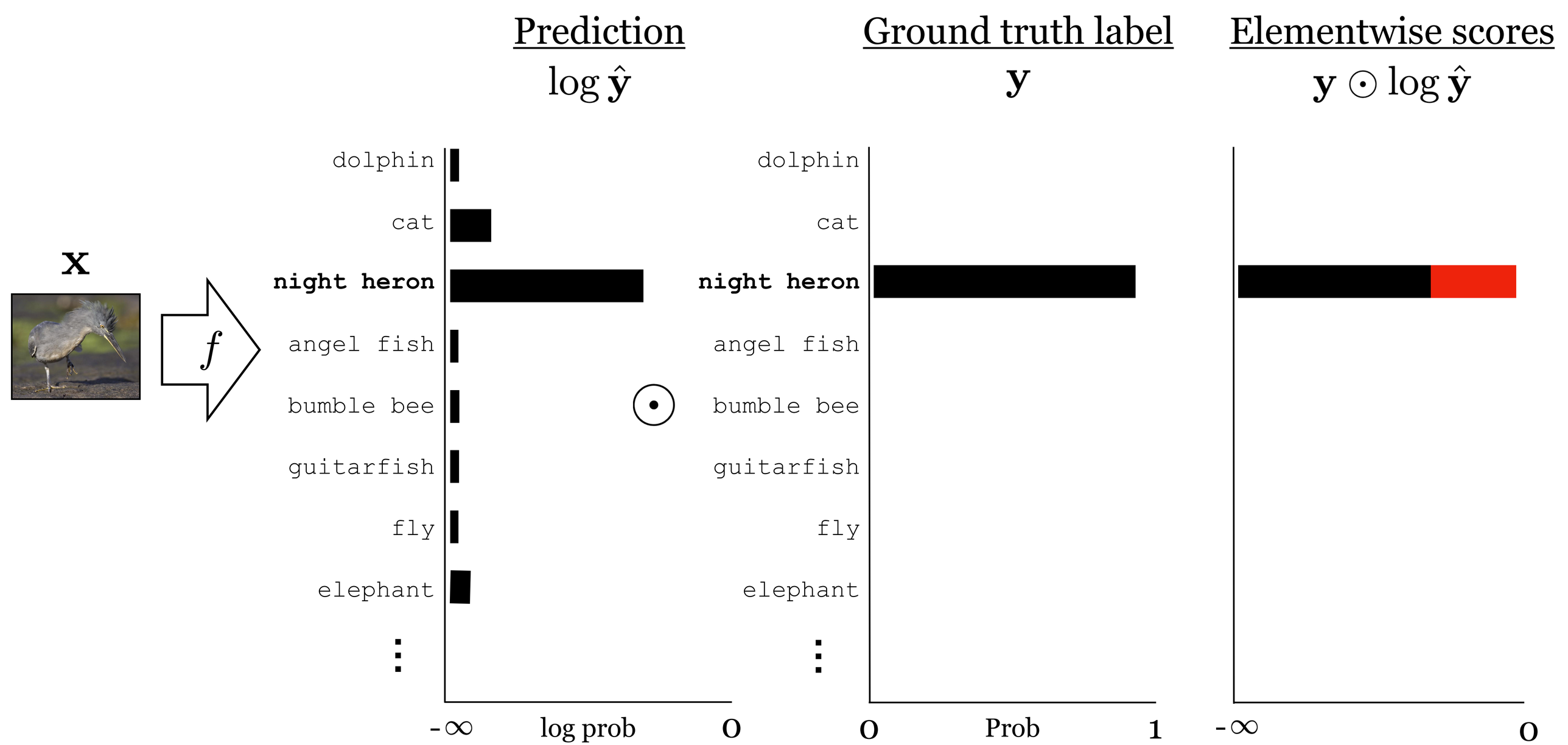
elephant

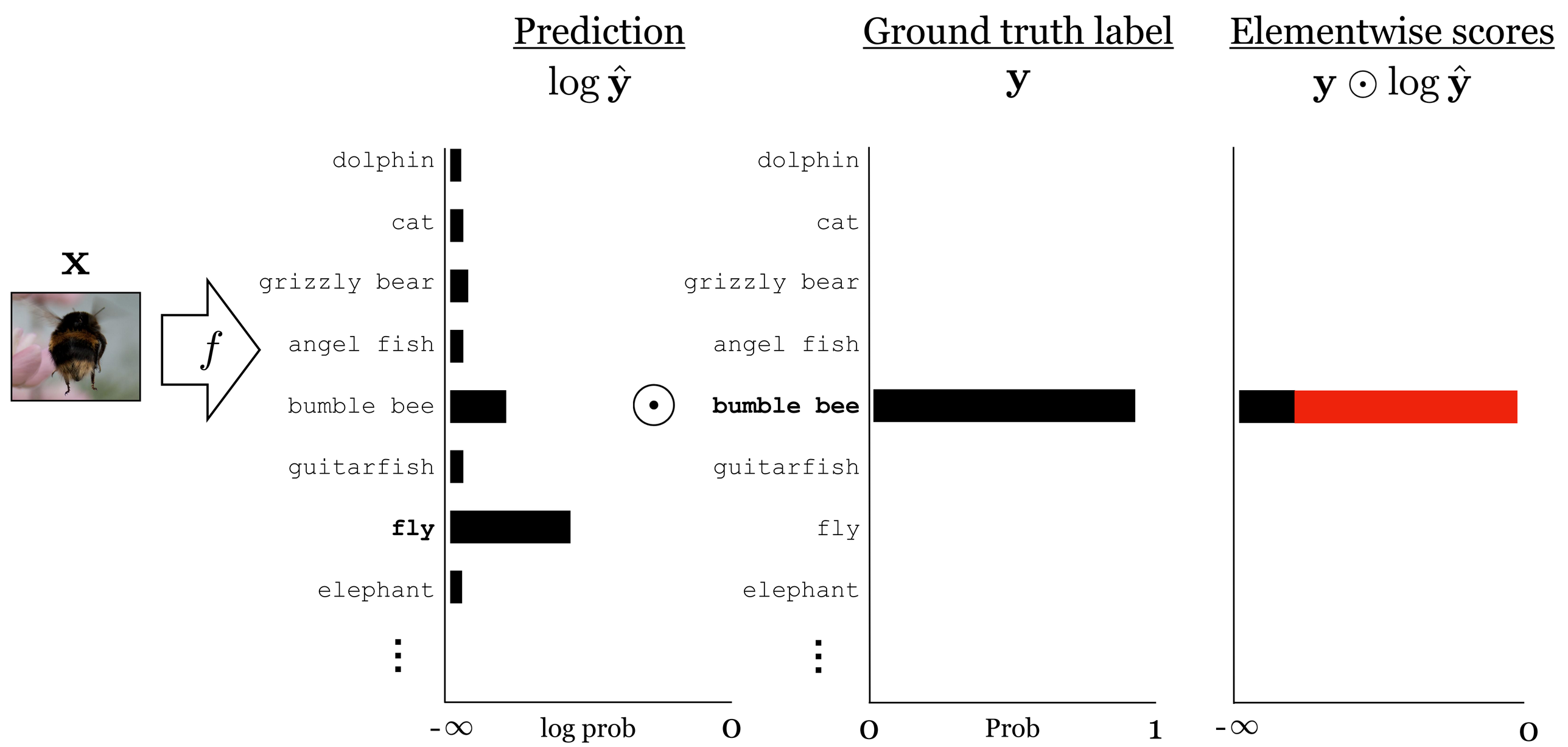
⋮

0 Prob 1


Prediction
log ŷ
Ground truth label
y
x
f
dolphin
cat
night heron
angel fish
bumble bee
guitarfish
fly
elephant
dolphin
cat
night heron
angel fish
bumble bee
guitarfish
fly
elephant
-∞
log prob
0
0
Prob
1


Prediction
log ŷ
Ground truth label
y
Elementwise scores
y ⊙ log ŷ
x
f
dolphin
cat
night heron
angel fish
bumble bee
guitarfish
fly
elephant
How much better you could have done
-∞
log prob
0
0
Prob
1
-∞
0


Prediction
log ŷ
Ground truth label
y
Elementwise scores
y ⊙ log ŷ
x
f
dolphin
cat
night heron
angel fish
bumble bee
guitarfish
fly
elephant
-∞
log prob
0
0
Prob
1
-∞
0


Prediction
log ŷ
Ground truth label
y
Elementwise scores
y ⊙ log ŷ
x
f
dolphin
cat
grizzly bear
angel fish
bumble bee
guitarfish
fly
elephant
⊙
-∞
log prob
0
0
Prob
1
-∞
0

Softmax regression (a.k.a. multinomial logistic regression)

$$f_\theta : \mathcal{X} \to \triangle^{K-1}$$

$\mathbf{z} = z_\theta(\mathbf{x})$ ⟵ logits: vector of K scores, one for each class

$\hat{\mathbf{y}} = \texttt{softmax}(\mathbf{z})$ ⟵ squash into a non-negative vector that sums to 1 — i.e. a probability mass function!

$$\hat{y}_j = \frac{e^{-z_j}}{\sum_{k=1}^{K} e^{-z_k}}$$

$\hat{\mathbf{y}} = f_\theta(\mathbf{x}) = \texttt{softmax}(z_\theta(\mathbf{x}))$

$\hat{\mathbf{y}} =$

Softmax regression (a.k.a. multinomial logistic regression)

Probabilistic interpretation:

$$\hat{\mathbf{y}} \equiv [P_\theta(Y = 1|X = \mathbf{x}), \ldots, P_\theta(Y = K|X = \mathbf{x})]$$ ⟵ predicted probability of each class given input x

$$H(\mathbf{y}, \hat{\mathbf{y}}) = -\sum_{k=1}^{K} y_k \log \hat{y}_k$$ ⟵ picks out the -log likelihood of the ground truth class $\mathbf{y}$ under the model prediction $\hat{\mathbf{y}}$

$$f^* = \arg\min_{f \in \mathcal{F}} \sum_{i=1}^{N} H(\mathbf{y}^{(i)}, \hat{\mathbf{y}}^{(i)})$$ ⟵ max likelihood learner!

Softmax regression (a.k.a. multinomial logistic regression)

Data

$$\{\mathbf{x}^{(i)}, \mathbf{y}^{(i)}\}_{i=1}^{N} \longrightarrow$$

Softmax regression

Objective

$$\mathcal{L}(f_\theta(\mathbf{x}), \mathbf{y}) = H(\mathbf{y}, f_\theta(\mathbf{x}))$$

Hypothesis space

$$f_\theta : \mathcal{X} \rightarrow \triangle^{K-1}$$

$$\longrightarrow \quad f_\theta$$

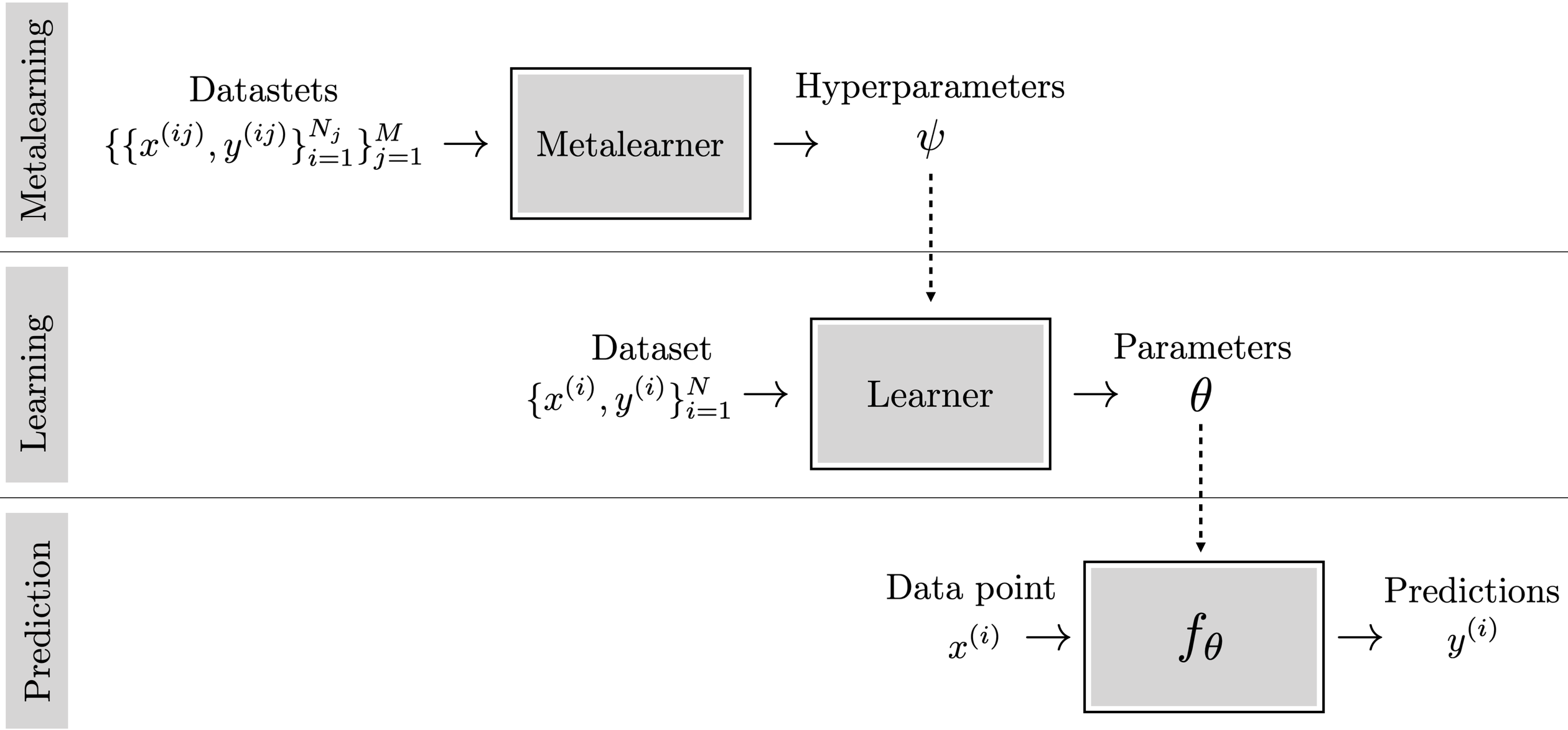
Learning to learn

(aka metalearning)

$$\{\texttt{input}: \big(x: [1,2], y: [1,2]\big), \quad \texttt{output}: y = x\}$$

$$\{\texttt{input}: \big(x: [1,2], y: [2,4]\big), \quad \texttt{output}: y = 2x\}$$

$$\{\texttt{input}: \big(x: [1,2], y: [0.5,1]\big), \quad \texttt{output}: y = \frac{x}{2}\}$$


Metalearning
Datastets
$\{\{x^{(ij)}, y^{(ij)}\}_{i=1}^{N_j}\}_{j=1}^{M}$
Metalearner
Hyperparameters
ψ
Learning
Dataset
$\{x^{(i)}, y^{(i)}\}_{i=1}^{N}$
Learner
Parameters
θ
Prediction
Data point
$x^{(i)}$
f_θ
Predictions
$y^{(i)}$

Summary

- Learning turns data into algorithms
- In this era of big data, it is a major component of almost all modern computer vision systems
- Formulation of computer vision tasks as (supervised) machine learning problems
- From linear regression to multi-class learning